



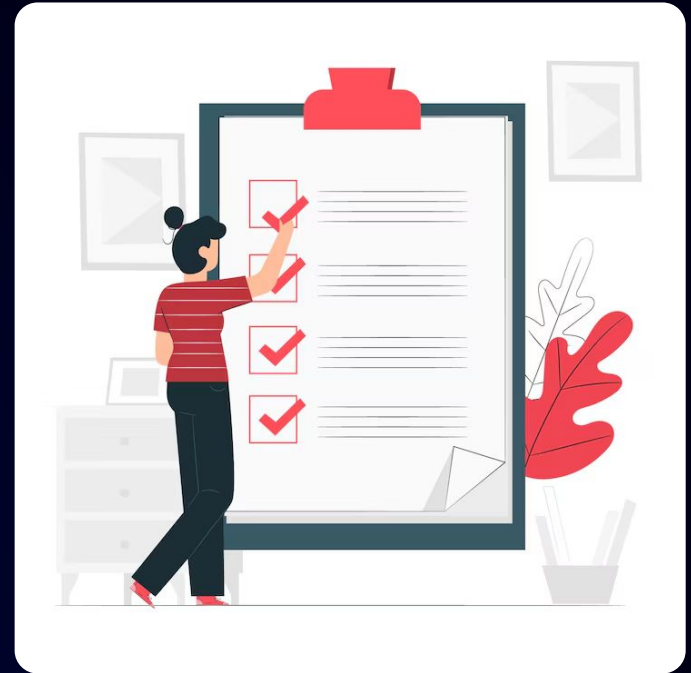
JavaScript Control Flow & Operators

Learning Objectives

- Understand how to use assignment operators to update values efficiently.
- Learn to apply logical operators (&&, ||, !) to combine conditions.
- Identify and use unary operators (++ , --) for quick value changes.
- Understand type conversion (implicit and explicit) and its impact on outputs.
- Use if-else statements to control program flow based on conditions.
- Apply nested conditionals for multi-level decision making.
- Implement if-else-if ladder for handling multiple scenarios.
- Use switch statements for cleaner handling of fixed-value conditions.
- Develop the ability to write clear, logical, and efficient conditional code.

Prerequisites

- Basic understanding of variables and data types (int, string, etc.)
- Familiarity with basic syntax of a programming language
- Knowledge of operators like +, -, *, /
- Understanding of simple input and output
- Basic idea of conditions (true/false values)

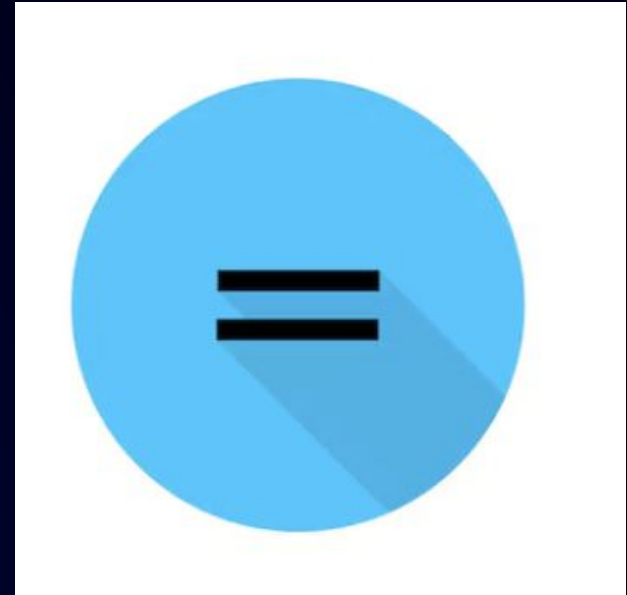




Assignment Operators in JavaScript

Understanding = (Basic Assignment):

- Used to **assign a value** to a variable
- Stores data from **right-hand side to left-hand side**
- Example: `let x = 10;`
- The variable on the left **receives and holds** the value



Using Compound Assignment Operators

Intro: These operators provide a short and efficient way to update variable values by combining operations with assignment.

Types of compound assignment operators

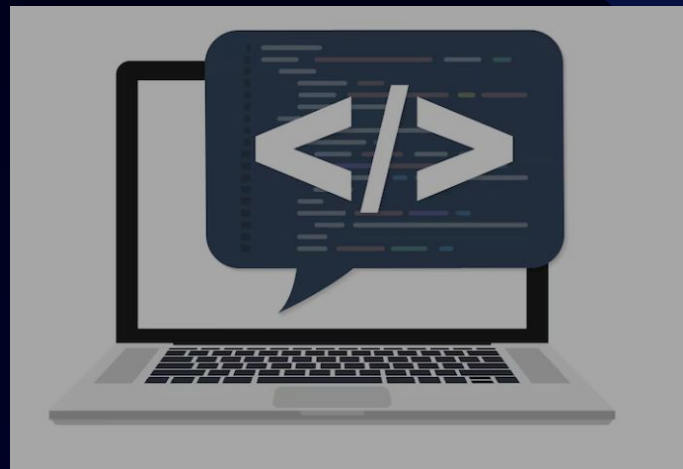
- `+=` : Adds and assigns ($x += 5 \rightarrow x = x + 5$)
- `-=` : Subtracts and assigns ($x -= 2$)
- `*=` : Multiplies and assigns ($x *= 3$)
- `/=` : Divides and assigns ($x /= 2$)



Quick Code Examples:

```
let x = 10; //assign initial value  
x += 5; //Adds 5 to x → x becomes 15  
x -= 3; //Subtracts 3 → x becomes 12  
x *= 2; //Multiplies by 2 → x becomes 24  
x /= 4; //Divides by 4 → x becomes 6
```

Note: Each step updates the same variable (x)



Real-World Use Cases :

Score Update (Games):

`score += 10;` → increases score

Shopping Cart:

`total += price;` → adds item price

Bank Balance:

`balance -= amount;` → deducts money

Counter (Loops):

`count += 1;` → increments value

Discount:

`price -= discount;` → reduces price



Pop Quiz

The += operator in Java adds a value to a variable. True or False?

A

True

B

False

Pop Quiz

The += operator in Java adds a value to a variable. True or False?

A

True

B

False



Logical Operators

Why Use Multiple Conditions in Programs?

- Real-world decisions often depend on **multiple factors**.
- Combining conditions helps to **make code more precise**.
- Avoids writing **nested if-else** statements, **keeping code cleaner and easier to read**.



Introduction : Logical Operators

- **Logical operators** are used to combine or modify conditions in a program.
- They help in **making decisions based on** multiple conditions.
- Common logical operators include **AND (&), OR (||), and NOT (!)**.
- These operators are widely used in real-life scenarios like login checks, validations, and eligibility conditions.



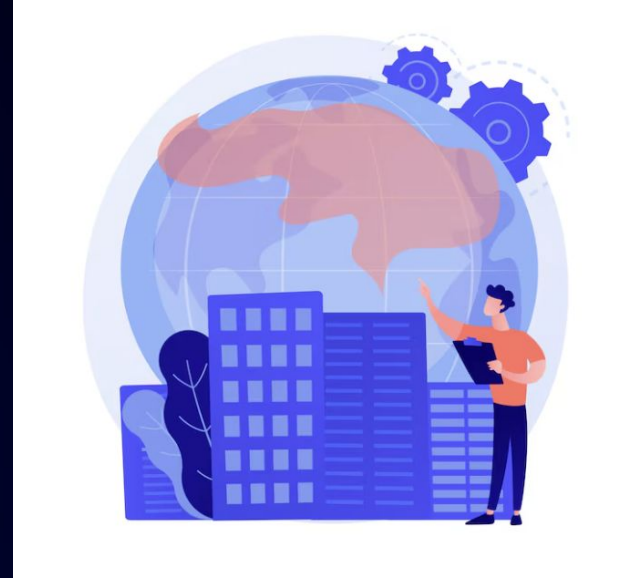
Types of Logical Operators :

- **Logical AND (&&) :** True only if all conditions are true. **Example:** Login validation username correct && password correct → allow access.
- **Logical OR (||) :** True if at least one condition is true. **Example:** Age + eligibility age ≥ 18 || has parental consent → can register.
- **Logical NOT (!) :** Reverses the condition (true → false, false → true). **Example:** !Blocked → allow login.



Real-World Use Cases :

- **isLoggedIn && isVerified** : allow access (both conditions must be true)
- **hasCoupon || isPremiumUser** : apply discount (any one condition is enough)
- **!isBlocked** : allow login (reverses condition: user should not be blocked)



Pop Quiz

Logical AND (&&) operator returns true only when both conditions are true. True or False?

A

False

B

True

Pop Quiz

Logical AND (&&) operator returns true only when both conditions are true. True or False?

A

False

B

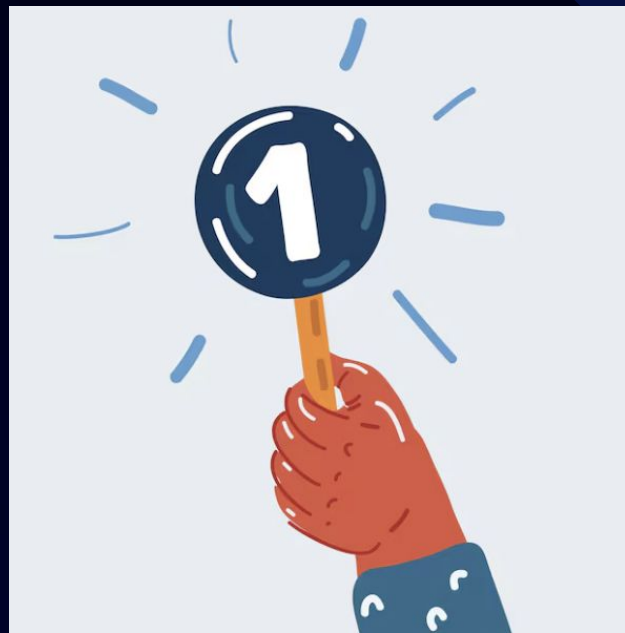
True



Unary Operators

What Are Unary Operators?

- Operators that work on a single variable/value.
- Used for increasing or decreasing values quickly.
- They are commonly used in loops, counters, and simple updates.
- Unary operators make code shorter and more efficient by avoiding longer expressions.



Types of Unary Operators :

Increment Operator (++) – Increase Value

Increases the value of a variable by 1.

Shortcut for: $x = x + 1$

Example:

```
int x = 5;
```

```
x++; → value becomes 6
```

Decrement Operator (--) – Decrease Value

Decreases the value of a variable by 1.

Shortcut for: $x = x - 1$

Example:

```
int x = 5;
```

```
x--; → value becomes 4
```



Prefix vs Postfix – Order Matters

Prefix ($++x$ / $--x$) :

Value is updated first, then used.

Example:

```
int x = 5;  
int y = ++x; → x = 6, y = 6
```

Postfix ($x++$ / $x--$) :

Value is used first, then updated.

Example:

```
int x = 5;  
int y = x++; → y = 5, x = 6
```



real-life examples :

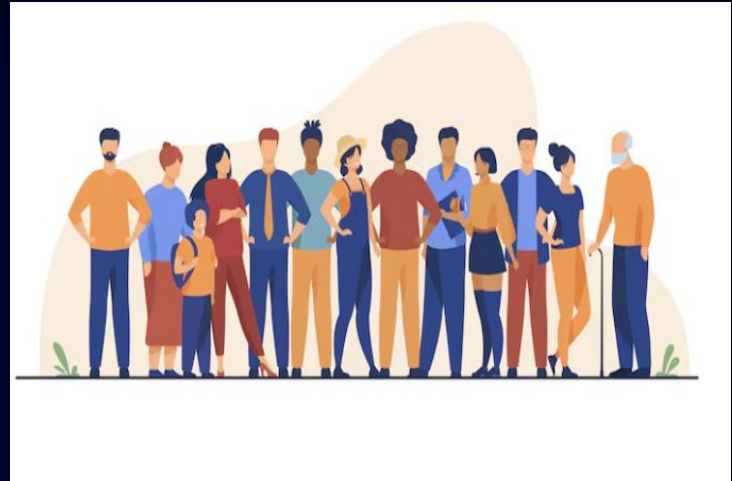
Increment (++) Examples :

Counting people entering a room:
`count++` : increases count by 1 for each person

Increasing score in a game:
`score++` : score goes up by 1

Decrement (--) Examples :

People leaving a room:
`count--` : decreases count by 1
Reducing lives in a game:
`lives--` : one life is lost



Pop Quiz

What will be the value of y? → `int x = 5; int y = x++;`

A

5

B

6

Pop Quiz

What will be the value of y? → `int x = 5; int y = x++;`

A

5

B

6



Understanding Type Conversion in Programming

Why Do We Need Type Conversion?

- To perform operations between **different data types** (e.g., number + string).
- To avoid **errors or unexpected results**.
- Makes programs more **flexible and adaptable**.



Automatic Conversion (Implicit)

- Done **automatically** by the **system**.
- No extra code required.
- Usually converts to a **compatible** or **larger type**.
- Example:
`5 + 2.5 → 7.5 (int becomes double automatically)`



Manual Conversion (Explicit)

- Done **by the programmer** using functions.
- Used **when automatic conversion may not give the desired result.**
- **Common methods :**

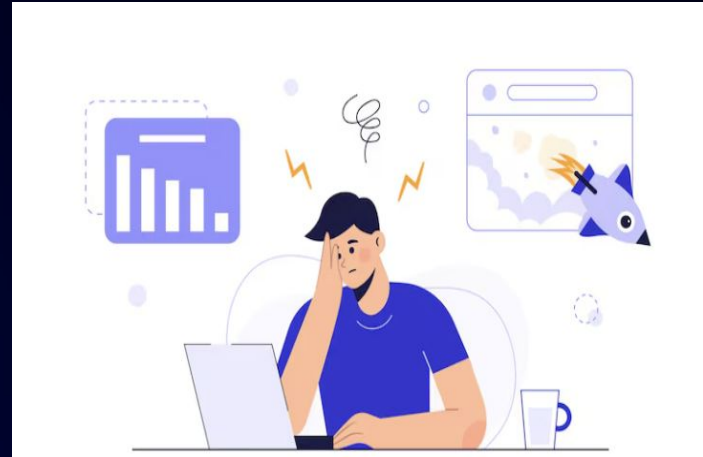
Number() : converts to number

String() : converts to string



Common Confusion Examples

- $"5" + 2 \rightarrow "52"$ (string + number $\rightarrow$ string concatenation)
- $"5" - 2 \rightarrow 3$ (string converted to number for subtraction)



Pop Quiz

Which type of conversion is done automatically by the system?

A

Explicit conversion

B

Implicit conversion

Pop Quiz

Which type of conversion is done automatically by the system?

A

Explicit conversion

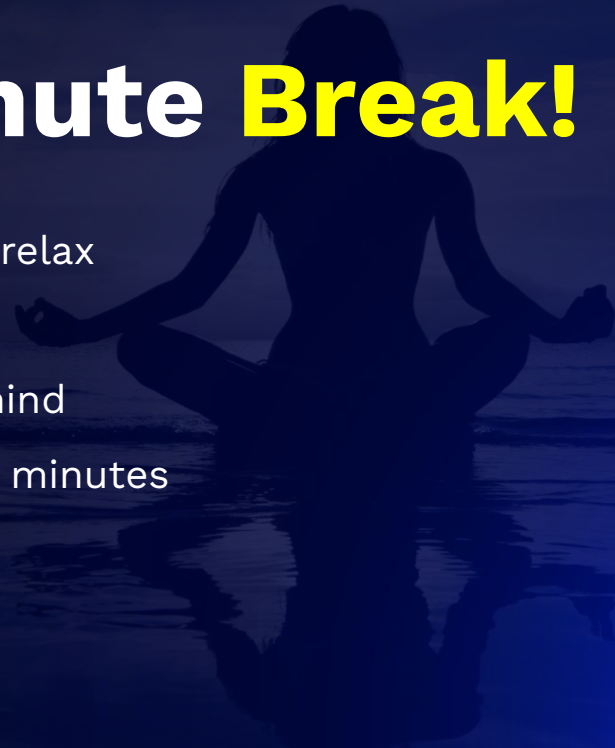
B

Implicit conversion



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

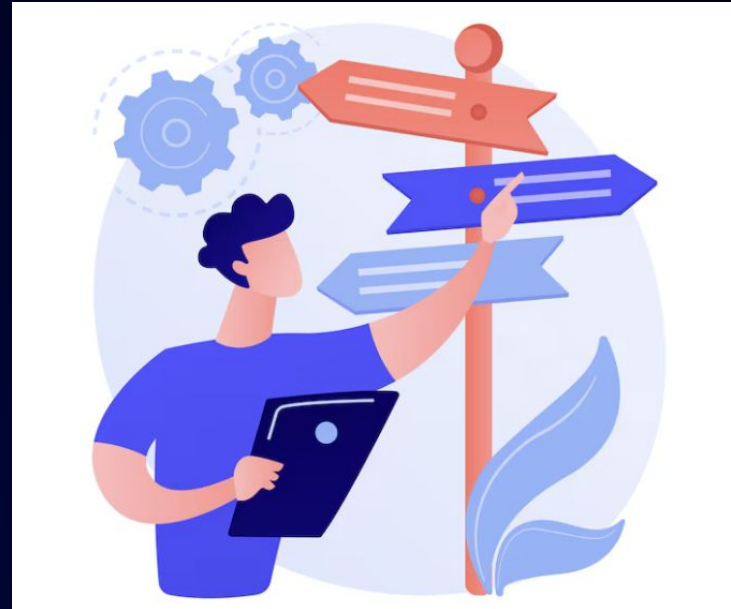




If/Else Statements

Making Decisions in Code

- Programs often need to **take decisions based on conditions**.
- Conditional statements help control the **flow of execution**.
- Code runs differently depending on whether a **condition is true or false**.

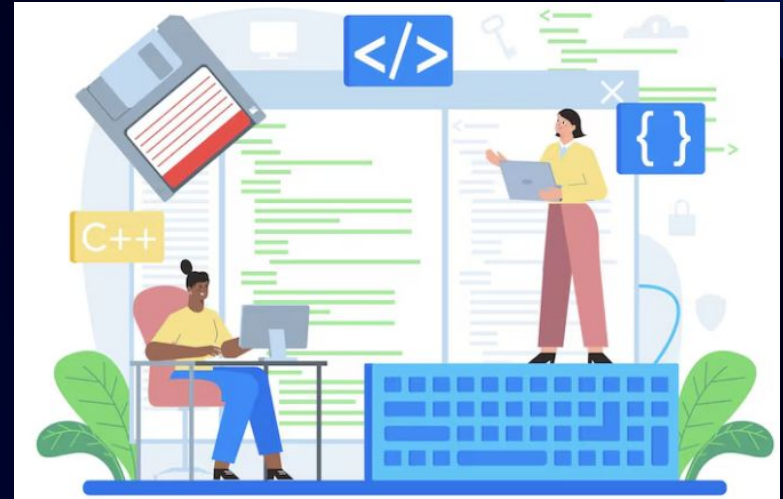


Structure of If-Else Statement

Used to check a condition and execute code accordingly.

Syntax:

```
if (condition) {
// code when condition is true
} else {
// code when condition is false
}
```



Applying If-Else in Scenarios

Using If-Else: Even or Odd Check

Check if a number is divisible by 2.
If condition is true : number is even.
Else: number is odd.

Using If-Else: Pass or Fail Logic

Check if marks are above passing criteria.
If condition is true : pass.
Else : fail.



Pop Quiz

Which block executes when the condition in an if-else statement is false?

A

if block

B

else block

Pop Quiz

Which block executes when the condition in an if-else statement is false?

A

if block

B

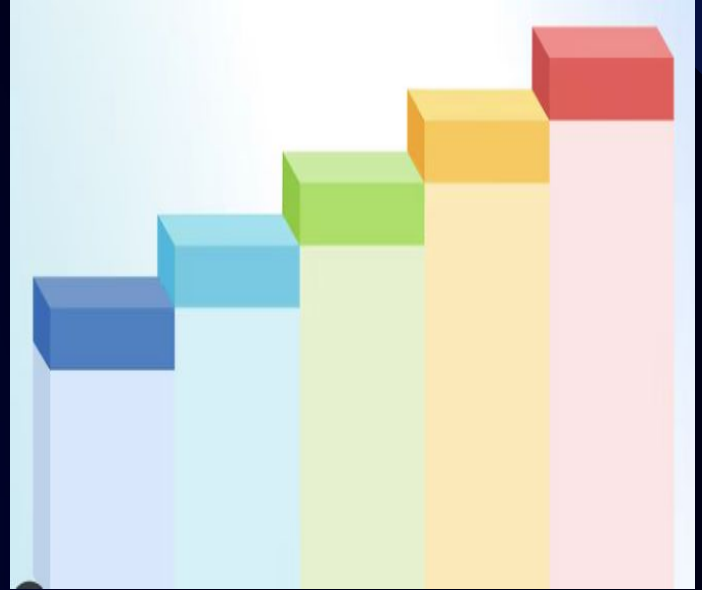
else block



Nested Conditionals

Handling Multiple Levels of Decisions

- Sometimes a single condition is not enough to make a decision.
- We can place one condition **inside another condition**.
- This helps in handling **step-by-step decision making**.
An if statement can be written **inside another if/else block**.
- The inner condition is checked **only if the outer condition is true**.
- Helps in creating **more specific and controlled logic**.



Scenario-Based Example: Login System

- **First check:** Is user logged in (valid username & password)?
- **If yes** → check user role (admin/user) for access level
- **If not** → deny access and show error



Pop Quiz

In nested conditionals, when does the inner condition execute?

A

Always

B

Only when the outer condition is true

Pop Quiz

In nested conditionals, when does the inner condition execute?

A

Always

B

Only when the outer condition is true



If-Else-If Ladder

Handling Multiple Conditions in Sequence

- Used when there are **more than two possible outcomes**.
- Conditions are checked **one by one in order**.
- As soon as one condition is true, the **rest are skipped**.
- Ensures that only **one block of code executes at a time**.
- Conditions are evaluated from top to bottom. The order of conditions matters **for correct results**.
- Helps **avoid writing multiple separate if statements**.
- Makes code more organized and readable.



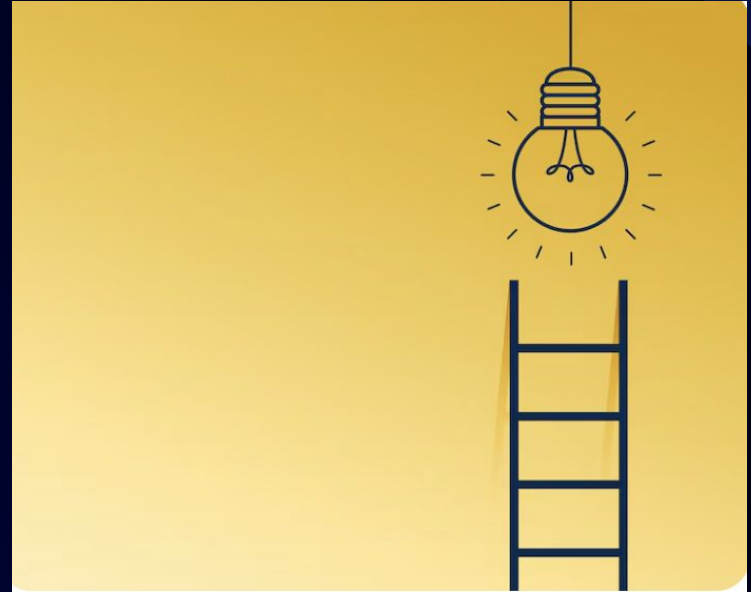
Understanding If-Else-If Structure

Helps in checking multiple conditions step-by-step.

Syntax:

```
if (condition1) {  
    // code  
} else if (condition2) {  
    // code  
} else {  
    // default code  
}
```

The else block runs when none of the above conditions are true.



Real-Life Example:

Grading System :

If marks $\geq 90 \rightarrow$ Grade A

Else if marks $\geq 75 \rightarrow$ Grade B

Else if marks $\geq 50 \rightarrow$ Grade C

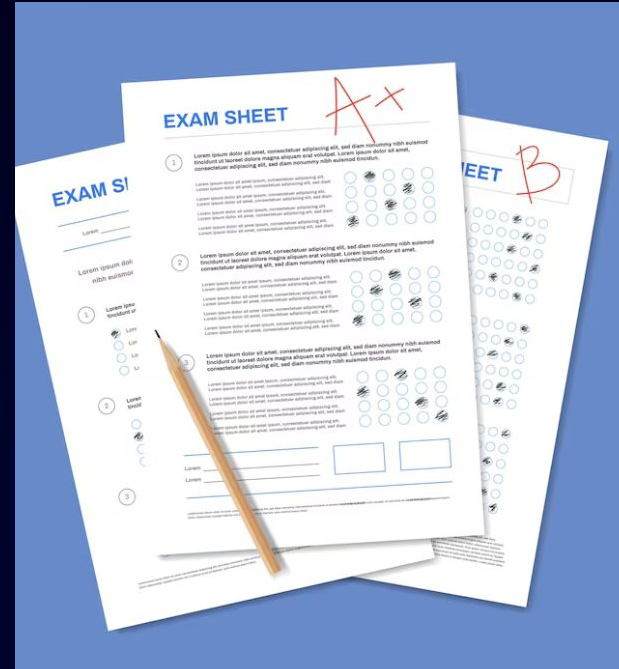
Else $\rightarrow$ Fail

Pricing Logic :

If amount $> 1000 \rightarrow$ High discount

Else if amount $> 500 \rightarrow$ Medium discount

Else $\rightarrow$ No discount



Pop Quiz

In an if-else-if ladder, what happens after one condition becomes true?

A

All conditions are checked

B

Remaining conditions are skipped

Pop Quiz

In an if-else-if ladder, what happens after one condition becomes true?

A

All conditions are checked

B

Remaining conditions are skipped



Switch Statement

A Cleaner Way to Handle Multiple Choices

- Used as an alternative to multiple if-else-if conditions.
- Best when checking a **single variable** against many fixed values.
- Makes code more **readable and organized**. Works best with **discrete (fixed) values** like numbers or characters.
- Each case represents a **specific value** to match.
- If break is not used, execution continues to the next case (fall-through).
- Helps reduce **long chains of if-else-if conditions**.
- Improves code clarity when many equal comparisons are needed.



Understanding Switch Syntax

- Compares a variable with different case values.

Syntax:

```
switch(value) {  
  case x:  
    // code  
    break;  
  default:  
    // code  
}
```

- break is used to stop execution after a match.
- default runs when no case matches.



Real-Life Example:

Example: Day of the Week

Input a number (1–7).

Each case represents a day.

Example:

1 → Monday

2 → Tuesday

Example: Menu Selection

User selects an option (1, 2, 3...).

Each case performs a specific action.

Example:

1 → View Profile

2 → Edit Profile

3 → Logout



Pop Quiz

What is the purpose of the break statement in a switch?

A

To check another condition

B

To exit the switch after a match

Pop Quiz

What is the purpose of the break statement in a switch?

A

To check another condition

B

To exit the switch after a match



Hands-On | Building the Grading System

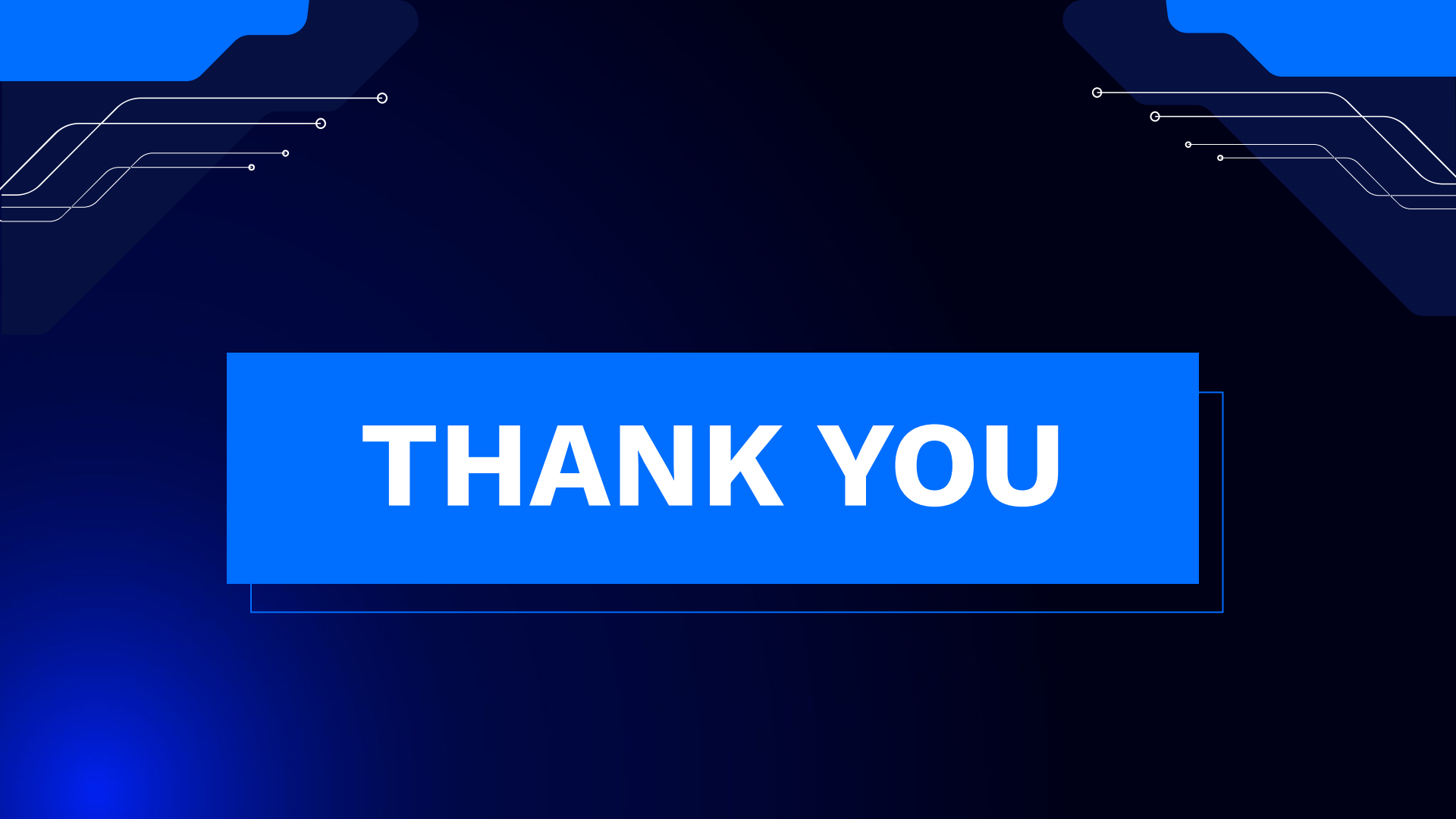
Summary

- Assignment Operators help update values quickly using shortcuts like +=, -=, etc.
- Logical Operators (&&, ||, !) are used to combine and modify conditions for better decision-making.
- Unary Operators (++ , --) work on a single value to increase or decrease it efficiently.
- Type Conversion allows changing data types (implicit or explicit) to ensure smooth operations.
- If-Else Statements help execute code based on true/false conditions.
- Nested Conditionals allow conditions inside conditions for more specific decisions.
- If-Else-If Ladder is used when there are multiple conditions to check in sequence.
- Switch Statement provides a cleaner way to handle multiple fixed choices.





DOUBT RESOLUTION



THANK YOU